Resumo – Matrizes

1. A ideia central

Uma lista resolve o problema de guardar as 5 pontuações de **um** jogador. Mas e se forem 3 jogadores, cada um com 5 pontuações? Criar `jogador1`, `jogador2`, `jogador3` repete código e não escala para 100 jogadores.

A solução é agrupar as listas dentro de outra lista. Essa "coleção de coleções" tem nome: **matriz**, uma estrutura organizada em **linhas** e **colunas** (bidimensional).

```
pontuacoes = [
    [10, 20, 30, 40, 50],
    [15, 25, 35, 45, 55],
    [5, 10, 15, 20, 25]
]

print(pontuacoes[0])
print(len(pontuacoes))
print(len(pontuacoes[0]))
```

```
[10, 20, 30, 40, 50]
3
5
```

Macete

No código, **matriz = lista de listas**. Cada **linha** é uma lista, e a matriz é a lista principal. Por isso `len(matriz)` é a quantidade de **linhas** e `len(matriz[0])` é a quantidade de **colunas**.

Usamos matrizes sempre que os dados têm duas dimensões: planilhas, tabuleiros e mapas de jogos, imagens (pixels), notas de alunos por disciplina.

2. Índices: matriz[i][j]

	j=0	j=1	j=2	j=3	j=4
i=0	10	20	30	40	50
i=1	15	25	35	45	55
i=2	5	10	15	20	25

```
pontuacoes = [
    [10, 20, 30, 40, 50],
    [15, 25, 35, 45, 55],
    [5, 10, 15, 20, 25]
]
```

```python
print(pontuacoes[0][0])
print(pontuacoes[1][3])
print(pontuacoes[2][4])
```

```
10
45
25
```

Macete

Leia `matriz[i][j]` como duas perguntas em ordem: `i` responde **"qual linha?"** e `j` responde **"qual coluna?"**. Primeiro a linha, depois a coluna. Uma forma de decorar: `matriz[i]` já é uma lista inteira (a linha), e o `[j]` escolhe um valor dentro dela.

Assim como nas listas, os índices começam em 0. O primeiro índice indica o "grupo" e o segundo indica o valor dentro do grupo: estamos acessando dados em **dois níveis**.

3. Percorrendo uma matriz

Sem índices (por elemento)

Um `for` só percorre as **linhas**; cada linha é uma lista. Para chegar em cada valor, é preciso um segundo `for`.

```python
pontuacoes = [[10, 20], [15, 25]]

for jogador in pontuacoes:
    print(jogador)

for jogador in pontuacoes:
    for ponto in jogador:
        print(ponto)
```

```
[10, 20]
[15, 25]
10
20
15
25
```

Com índices

```python
matriz = [[10, 20], [15, 25]]

for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        print(i, j, matriz[i][j])
```

```
0 0 10
0 1 20
1 0 15
1 1 25
```

Observe a ordem da saída: o `i` "trava" em 0 enquanto o `j` dá a volta completa (0, 1); só então o `i` passa para 1 e o `j` **recomeça do zero**.

Macete

O laço de dentro gira **inteiro** para cada volta do laço de fora, como os ponteiros de um relógio: o `j` é o ponteiro dos minutos e o `i` é o das horas. Com o laço das linhas por fora, a matriz é visitada na ordem de leitura de um texto: da esquerda para a direita, de cima para baixo.

Atenção

O professor avisa nos slides: **a grande maioria dos algoritmos clássicos com matrizes trabalha com índices**. Acostume-se a manipular `i` e `j`. Use o percurso sem índice apenas quando a posição realmente não importar.

4. Construindo uma matriz durante a execução

O raciocínio é o mesmo das listas, em dois níveis: (1) criar a lista principal; (2) para cada linha, criar uma **nova** lista; (3) preencher essa lista; (4) inserir a linha na estrutura principal.

```
pontuacoes = []

for i in range(3):
    jogador = []

    for j in range(5):
        ponto = int(input())
        jogador.append(ponto)

    pontuacoes.append(jogador)
```

Atenção

A linha `jogador = []` precisa ficar **dentro** do `for` externo. Se ela ficar fora, todos os valores são empilhados na mesma lista e a matriz sai errada. Regra: **linha nova a cada volta do laço de fora**.

5. Os padrões que resolvem quase tudo

Nos exemplos desta seção, considere sempre a matriz:

$$\begin{pmatrix} 2 & 5 & 1 \\ 4 & 3 & 6 \\ 7 & 2 & 8 \end{pmatrix}$$

Padrão 1: um resultado para a matriz inteira

Soma total, maior valor, contagem de pares: o acumulador é criado **antes dos dois laços**.

```
matriz = [[2, 5, 1], [4, 3, 6], [7, 2, 8]]

soma = 0
maior = matriz[0][0]
```

```
pares = 0

for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        soma += matriz[i][j]

        if matriz[i][j] > maior:
            maior = matriz[i][j]

        if matriz[i][j] % 2 == 0:
            pares += 1

print(soma, maior, pares)
```

```
38 8 5
```

Padrão 2: um resultado por linha

Soma ou média de cada linha: o acumulador é zerado **dentro do laço externo**, e o resultado é usado depois que o laço interno termina.

```
matriz = [[2, 5, 1], [4, 3, 6], [7, 2, 8]]

for i in range(len(matriz)):
    soma = 0

    for j in range(len(matriz[i])):
        soma += matriz[i][j]

    print("Linha", i, soma)
```

```
Linha 0 8
Linha 1 13
Linha 2 17
```

Macete

Onde zerar o acumulador? É a pergunta que mais derruba em matrizes. Resultado da matriz inteira: zera **fora** dos dois laços. Resultado por linha (ou por coluna): zera **entre** os dois laços. O `print` ou o `append` do resultado fica alinhado com o `for` de dentro, não dentro dele.

Padrão 3: um resultado por coluna

Basta **inverter os laços**: a coluna `j` vai para fora e a linha `i` para dentro. O acesso continua sendo `matriz[i][j]`.

```
matriz = [[2, 5, 1], [4, 3, 6], [7, 2, 8]]
somas = []

for j in range(len(matriz[0])):
    soma = 0

    for i in range(len(matriz)):
        soma += matriz[i][j]
```

```
    somas.append(soma)

print(somas)
```

```
[13, 10, 15]
```

Macete

Trocar a ordem dos laços **não** troca a ordem dos colchetes. É sempre `matriz[linha][coluna]`. O que muda é quem fica "travado" enquanto o outro gira: para somar uma coluna, o `j` fica travado e o `i` desce.

Padrão 4: diagonal principal e matriz quadrada

Uma matriz é **quadrada** quando tem o mesmo número de linhas e colunas. Na **diagonal principal**, linha e coluna são iguais (`i == j`). Como só existe um elemento da diagonal por linha, um único laço basta.

```
matriz = [[2, 5, 1], [4, 3, 6], [7, 2, 8]]

quadrada = len(matriz) == len(matriz[0])
soma = 0

for i in range(len(matriz)):
    soma += matriz[i][i]

print(quadrada, soma)
```

```
True 13
```

Macete

Mapa das posições em uma matriz quadrada: `i == j` é a diagonal principal; `j > i` é a região **acima** dela; `j < i` é a região **abaixo**. A matriz identidade é só isso: 1 onde `i == j` e 0 no resto.

Padrão 5: construir uma matriz por regra (identidade)

```
n = 3
identidade = []

for i in range(n):
    linha = []

    for j in range(n):
        if i == j:
            linha.append(1)
        else:
            linha.append(0)

    identidade.append(linha)

for linha in identidade:
    print(linha)
```

```
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]
```

Padrão 6: melhor linha (ou coluna)

Combina o padrão 2 com "maior valor": calcule a soma de cada linha e guarde a posição da melhor.

```python
matriz = [[2, 5, 1], [4, 3, 6], [7, 2, 8]]

melhor = 0
maior_soma = sum(matriz[0])

for i in range(1, len(matriz)):
    soma = sum(matriz[i])

    if soma > maior_soma:
        maior_soma = soma
        melhor = i

print(melhor, maior_soma)
```

```
2 17
```

Aqui `sum(matriz[i])` funciona porque `matriz[i]` é uma lista comum. Se a função for proibida, substitua pelo laço interno do padrão 2. O sinal `>` (e não `>=`) garante que, em caso de empate, vale a **primeira** linha encontrada.

6. Erros mais comuns

Atenção

- **Trocar linha por coluna:** escrever `matriz[j][i]` em uma matriz que não é quadrada causa `IndexError`, ou pior, funciona e dá a resposta errada em uma quadrada.
- **Confundir os tamanhos:** `len(matriz)` conta linhas; `len(matriz[0])` conta colunas. Em uma matriz 2×3, o `i` vai de 0 a 1 e o `j` de 0 a 2.
- **Criar a linha fora do laço:** `linha = []` antes do `for` externo.
- **Zerar o acumulador no lugar errado:** zerar a soma da linha fora dos dois laços faz as somas se acumularem de uma linha para a outra.
- **Esquecer que a função altera a matriz:** uma função que faz `matriz[i][j] = ...` modifica a matriz original do programa principal.
- **Matriz vazia:** os casos de teste dos slides incluem `[[]]`. Antes de dividir pela quantidade de elementos, verifique se ela é maior que zero.

7. Questões de exemplo resolvidas

Questão 1 — Média por jogador (desafio 1 dos slides)

As pontuações estão em uma matriz em que cada linha é um jogador e cada coluna é uma partida. Calcule a média de cada jogador e exiba no formato `Media do jogador 1: X pontos`.

Raciocínio. Um resultado por linha: padrão 2. Atenção a dois detalhes: o jogador 1 está na linha 0 (por isso `i + 1`) e um jogador sem partidas não pode causar divisão por zero.

```
def exibir_medias(matriz):
    for i in range(len(matriz)):
        soma = 0

        for j in range(len(matriz[i])):
            soma += matriz[i][j]

        if len(matriz[i]) > 0:
            media = soma / len(matriz[i])
        else:
            media = 0

        print("Media do jogador", i + 1, ":", media, "pontos")


exibir_medias([[1, 2, 3], [4, 5, 6]])
exibir_medias([[]])
```

```
Media do jogador 1 : 2.0 pontos
Media do jogador 2 : 5.0 pontos
Media do jogador 1 : 0 pontos
```

Questão 2 — Jogadores acima da média geral (desafio 2 dos slides)

Calcule a média de cada jogador, a média geral e exiba quais jogadores tiveram média acima da média geral.

Raciocínio. Três etapas: (1) guardar a média de cada linha em uma lista; (2) calcular a média dessa lista; (3) filtrar. É o problema de "acima da média" das listas, aplicado sobre as médias.

```
matriz = [[1, 2, 3], [4, 5, 6]]
medias = []

for i in range(len(matriz)):
    soma = 0

    for j in range(len(matriz[i])):
        soma += matriz[i][j]

    medias.append(soma / len(matriz[i]))

geral = sum(medias) / len(medias)
acima = []

for i in range(len(medias)):
```

```
    if medias[i] > geral:
        acima.append(i + 1)

print(medias, geral)
print("Jogadores acima da media:", acima)
```

```
[2.0, 5.0] 3.5
Jogadores acima da media: [2]
```

Questão 3 — Mapa de jogo (desafio 3 dos slides)

O mapa é uma matriz de pontos. O jogador começa no canto superior esquerdo, termina no canto inferior direito e só pode andar para a **direita** ou para **baixo**. Qual é o maior número de pontos que ele pode coletar?

Raciocínio. Testar todos os caminhos é inviável. A ideia é responder, para **cada célula**, "qual é o máximo de pontos com que consigo chegar aqui?". Só existem duas formas de chegar a uma célula: vindo de **cima** ou vindo da **esquerda**. Então o melhor para a célula é o seu próprio valor mais o **maior** entre esses dois vizinhos. Na primeira linha só dá para vir da esquerda; na primeira coluna, só de cima.

```
def maior_caminho(mapa):
    if len(mapa) == 0 or len(mapa[0]) == 0:
        return 0

    melhor = []

    for i in range(len(mapa)):
        linha = []

        for j in range(len(mapa[i])):
            if i == 0 and j == 0:
                linha.append(mapa[i][j])
            elif i == 0:
                linha.append(mapa[i][j] + linha[j - 1])
            elif j == 0:
                linha.append(mapa[i][j] + melhor[i - 1][j])
            else:
                cima = melhor[i - 1][j]
                esquerda = linha[j - 1]

                if cima > esquerda:
                    linha.append(mapa[i][j] + cima)
                else:
                    linha.append(mapa[i][j] + esquerda)

        melhor.append(linha)

    for linha in melhor:
        print(linha)

    return melhor[len(mapa) - 1][len(mapa[0]) - 1]


print(maior_caminho([[5, 1, 3], [2, 10, 1], [1, 1, 20]]))
print(maior_caminho([[]]))
```

```
[5, 6, 9]
[7, 17, 18]
[8, 18, 38]
38
0
```

A resposta é o valor do canto inferior direito da matriz auxiliar: 38, obtido pelo caminho $5 \to 2 \to 10 \to 1 \to 20$.

Questão 4 — Leitura de código

Escreva a saída do programa abaixo.

```python
matriz = [[4, 1, 7], [2, 6, 3]]
total = 0

for i in range(len(matriz)):
    parcial = 0

    for j in range(len(matriz[i])):
        if (i + j) % 2 == 0:
            parcial = parcial + matriz[i][j]
        else:
            parcial = parcial - matriz[i][j]

    total = total + parcial
    print("Linha", i, parcial)

print("Total", total)
```

Raciocínio. Desenhe a matriz e marque o sinal de cada célula pela soma dos índices: par soma, ímpar subtrai. O resultado é um "tabuleiro de xadrez" de sinais.

	j=0	j=1	j=2	parcial
i=0	$+4$	-1	$+7$	10
i=1	-2	$+6$	-3	1

```
Linha 0 10
Linha 1 1
Total 11
```

8. Cola rápida

Quero...	**Escrevo...**
quantidade de linhas	`len(matriz)`
quantidade de colunas	`len(matriz[0])`
um elemento	`matriz[i][j]`
uma linha inteira	`matriz[i]`
primeira coluna	`matriz[i][0]`, variando `i`
diagonal principal	`matriz[i][i]`
é quadrada?	`len(matriz) == len(matriz[0])`
acima / abaixo da diagonal	`j > i` / `j < i`
percorrer por linhas	`for i` fora, `for j` dentro
percorrer por colunas	`for j` fora, `for i` dentro